

Engineering standards

Ten rules, each enforced by a machine rather than a person, and how to get a resistant team to adopt them

One idea underneath all of this: **if a standard matters, a machine enforces it. If no machine can enforce it, it is advice, and it should be written down as advice.** A rule that depends on someone remembering it under deadline pressure has a half-life of about six weeks.

The standards

Each one exists because of a specific finding in the [assessment](#).

#	STANDARD	ENFORCED BY	ON FAILURE
1	Formatting is automatic and never discussed	Prettier, pre-commit and CI	CI fails; fix is <code>npm run format</code>
2	No credential ever enters the repository	Secret scanning on push	Push rejected
3	Route handlers only do transport	Module boundary lint + review checklist	Review blocks; new routes only
4	Authorisation is checked in the service layer	Test required per service method	CI fails on an untested public method
5	Money paths are transactional and idempotent	Reconciliation job + code owners on billing	Nightly drift alert; second reviewer required
6	Every bug fix ships with the test that would have caught it	Pull request template + review	Review blocks
7	Coverage on changed lines never falls	Coverage ratchet in CI	CI fails
8	Everything reaches production through CI	Deploy keys removed from laptops	Impossible by construction
9	API responses are explicit objects, never raw rows	Type boundary + review checklist	Review blocks; new endpoints only
10	Security advisories patched within a week	Scheduled dependency PRs	Ticket auto-created, visible in standup

Three choices worth defending:

Rule 6 is not "write tests". "Write tests" is a value, and it loses to a deadline every time. "The bug fix includes a test reproducing the bug" is narrow, obviously reasonable, and lands at the one moment writing a test feels satisfying — you have just spent two hours finding the thing. Over a year it builds a suite concentrated exactly where the bugs actually are.

Rule 7 is a ratchet, not a target. No percentage, no backfilling. Coverage on lines you changed cannot be lower than before. Targets get gamed; a ratchet is just a door closing behind you.

Rules 3 and 9 apply to new and modified code only. Never retroactively.

Not standards

- **No branch naming or commit message convention.** I have opinions; they are not worth the goodwill.
- **No mandated frontend architecture.** The components are messy and they caused none of the nine findings.
- **No pull request size limit.** It teaches people to split changes in ways that are harder to review.
- **No "no `any` in TypeScript".** In a codebase this age that means a thousand suppressions. It becomes reasonable once the modules exist.

Getting a resistant team to adopt them

The team is not resistant because they are careless. They shipped a real business with four people and no runway, every shortcut was a rational trade at the time, and a new senior hire arriving with a quality document is a genre they have seen before.

What the resistance usually is

WHAT IT SOUNDS LIKE	WHAT IT USUALLY MEANS	WHAT HELPS
"We don't have time for this"	I am measured on delivery and this is a tax	Make the first changes visibly <i>save</i> them time
"That won't work here"	You don't understand our constraints yet	Be right about something hard first
"We tried that before"	An initiative was abandoned and they looked silly for investing	Start small, finish it, point at it
Silence, then nothing changes	This feels like criticism of my code	Never audit old code. Rules apply going forward
"Just this once, we need to ship"	The rule has no legitimate escape hatch	Give them one, cheap to use and visible

The sequence

Weeks 1–2: propose nothing. Do the week-1 work from the [migration plan](#) — error tracking, CI, the deploy pipeline, the rotation. All of it removes pain the team already feels: deploys stop depending on one person, and nobody learns about outages from a forwarded customer email. The person proposing standards should first be the person who deleted your worst chore.

Week 3: they write the list, I edit it. Ninety minutes, whole team, one question on the board: *what has wasted your time in the last month?* The list that comes back is always close to the one I would have written — flaky

deploys, fear of touching billing, three places to change a price. I add the two or three they did not raise (secret scanning, authorisation in the service layer) with evidence from the assessment. The output is mostly theirs, which produces better rules and means they survive me leaving.

Week 4: the mechanical rules. Formatting, secret scanning, CI. Unarguable, no judgment required, and each lands as tooling that already works rather than a rule to remember. The team gets to experience "a standard landed and it was fine" before anything harder arrives.

Weeks 5–8: the test rules. Rule 6 first, then the ratchet once there is something to ratchet from. By now the characterization tests exist on the money paths, so the example is in our codebase rather than a blog post from a company with 400 engineers.

Weeks 9–12: the architectural rules. Thin handlers, explicit responses, authorisation in services. Last, because by then the extracted modules exist and the rule is no longer abstract — it is "like the subscriptions module", which is a thing they can open.

Day 90: review with data, delete what is not working. Every rule is up for removal. If a standard has moved nothing and people resent it, it goes — and that is the most credible thing that can happen to the rest of them.

The tactics that matter most

Never audit existing code. Rules apply to what you touch. This removes the "we'd have to rewrite everything" objection, which is the one that kills these initiatives, and it is also correct: untouched code is not where the risk is.

Machines say no, people say why. I will never leave a review comment about formatting or a missing test that CI could have caught. Once a person is the enforcement mechanism, the standard becomes a relationship problem and people route around the reviewer rather than the rule.

A cheap, visible escape hatch. Any rule can be bypassed with `[skip-standard: reason]` in the pull request. It works immediately, no approval. Every use is reviewed monthly — not to shame anyone, but because a rule bypassed six times a month is badly designed and the bypasses are the bug report.

Model it before requiring it. My pull requests follow every rule from day one, including the ones not yet adopted, and I take the ugliest module rather than the clean greenfield work.

Convert the sceptic, do not overrule them. There is always one, usually the longest-tenured engineer, and they are usually right about something. Give them ownership of the standards they care most about and let them present those. Overruling them ends the initiative on a timer, because everyone else is watching to see who wins.

Timebox the debate. Two weeks to argue, then we decide, then we revisit at 90 days. An explicit revisit date is what makes it safe to stop arguing.

How I would know it is working

Not compliance rates, which measure fear.

- Someone quotes a standard back at me in a review of *my* code.
- Someone proposes a new one without being asked.

- The escape hatch is used two or three times a month. Fifteen means the rules are wrong; zero means people are ignoring them quietly.
- Change failure rate falls, from the [migration plan](#) numbers.

If at day 90 the rules only hold when I am in the review, that is evidence I got the sequencing wrong, not that the team is at fault. I would cut the list to the three that are pure automation — formatting, secret scanning, CI-only deploys — which need no consent to keep working. A small standard that holds beats a comprehensive one everyone has quietly agreed to ignore.